

Warm-up: Concurrency on One Thread

Programming Paradigms · Lab 6 · session 6: asynchronous programming

The lab follows the two blocks of the lecture: block 1, the introduction to Part 2, and block 2, asynchronous programming (concurrency on one thread).

The **warm-up** is the main part of the lab, and we finish it during the lab session. It has three questions on paper on the introduction; six programs in JavaScript, five to predict and one to repair; and three questions in Python, on two programs, to predict. We write each prediction down before we run the program. The **quest** is optional: five short exercises on continuations. They go with the appendix of block 2, *The suspended computation as a value*, which we read first. Nothing is handed in.

The programs are in `warmup/js/` and `warmup/py/`. The quest is on the lab's page, `index.html`.

Part 1: Correctness and coordination (on paper)

Three questions on the introduction. We write the answers on a page of our own.

From the lecture.

Safety: nothing bad ever happens; a violation happens at one definite moment. *Liveness*: something good eventually happens; no finite beginning of an execution violates it: however long the execution has run, the good thing can still happen later.

Coordination: when a process may act depends on what the others have done or are doing. It has two forms: *not before* (P waits until Q has acted) and *not at the same time* (P and Q are never both inside the part that changes the shared data).

Linearizability: in every execution, the calls can be put in one sequence such that each call answers as it would if the calls were made alone, one after the other, in that order; and a call that returned before another was made comes first in the sequence.

Question 1

Given the five requirements of the bank account below, say of each whether it is a safety property or a liveness property.

- (a) no deposit answered *ok* is lost;
- (b) no request changes the balance twice;
- (c) the balance is never negative;
- (d) every request gets a definitive answer in the end;
- (e) the two copies of the account, kept on two machines, agree in the end.

Question 2

A semaphore is a counter with two operations: **acquire** waits until the counter is above zero, then takes one off; **release** adds one. Given Dijkstra's bounded buffer in Java, as in the introduction, say for each of the three semaphores whether it expresses *not before* or *not at the same time*, and which action waits for which other action.

Java

```
Semaphore numberOfQueuingPortions = new Semaphore(0);
Semaphore numberOfEmptyPositions  = new Semaphore(N); // N places
Semaphore bufferManipulation      = new Semaphore(1);

// producer
for (int k = 1; k <= 8; k++) {
    String portion = produceNextPortion(k);
    numberOfEmptyPositions.acquire();
    bufferManipulation.acquire();
    buffer.addLast(portion);
    bufferManipulation.release();
    numberOfQueuingPortions.release();
}

// consumer
for (int k = 1; k <= 8; k++) {
    numberOfQueuingPortions.acquire();
    bufferManipulation.acquire();
    String portion = buffer.removeFirst();
    bufferManipulation.release();
    numberOfEmptyPositions.release();
    processPortionTaken(portion);
}
```

Question 3

Given the bank account with a balance of 100, where **withdraw(n)** answers *refused* when the balance would become negative, and two clients, A and B, that each call **withdraw(80)** at the same moment, so that the two calls overlap: for each of the two executions below, say whether its calls can be put in such a sequence, and if they can, give the sequence.

- (a) both calls answer *ok*;
- (b) A's call answers *ok*, B's call answers *refused*.

Part 2: Asynchronous functions in JavaScript (predict on paper, then run)

We read each program, choose one of the three answers, and write it down. Then we run the program in `warmup/js/`, for example `node q4_two_loads.mjs`. Where the run differs from our answer, we find the property that explains it. In question 9 we write a function, then run it.

From the lecture.

Property 1 · Suspension. An asynchronous function is suspended only at an `await` written in it. A caller that does not await the call goes on.

Property 2 · Waiting is not running. Waits overlap. On one thread, our own code does not: at any moment at most one computation is running.

Property 3 · Run-to-completion. The loop takes the next function from the queue only when the stack is empty: the running code has returned, and so has everything it called. Nobody takes the thread away from running code.

Property 4 · Atomicity between suspension points. A stretch of a function's text with no `await` in it is atomic with respect to all other code of ours: nobody observes it half done. A function called inside the stretch runs at once, inside its caller, and is part of the stretch.

In JavaScript, in addition, a call runs the function up to its first `await` and then returns a promise; and every `await` is a suspension point, even `await null`.

Question 4

Given the program below, what does it print, in which order?

JavaScript

`warmup/js/q4_two_loads.mjs`

```
// Question 4. Run: node q4_two_loads.mjs
// fetchUser answers "Ada" after 300 ms: a request, imitated by a timer
const fetchUser = () => new Promise(res => setTimeout(() => res("Ada"), 300));

async function load(who) {
  console.log("A", who);
  const user = await fetchUser();
  console.log("C", who, user);
}
load(1);
load(2);
console.log("B");
```

- (a) A 1, C 1 Ada, A 2, C 2 Ada, B
- (b) A 1, A 2, B, C 1 Ada, C 2 Ada
- (c) B, A 1, A 2, C 1 Ada, C 2 Ada

Question 5

Given the program below, in which the `await` is forgotten, what does it print, in which order?

JavaScript

warmup/js/q5_forgotten.mjs

```
// Question 5. Run: node q5_forgotten.mjs
// fetchUser answers "Ada" after 300 ms: a request, imitated by a timer
const fetchUser = () => new Promise(res => setTimeout(() => res("Ada"), 300));

async function load() {
  console.log("A");
  const user = fetchUser();           // the await is forgotten
  console.log("C", user);
}
load();
console.log("B");
```

- (a) A, B, C Ada
- (b) A, C Ada, B
- (c) A, C Promise { <pending> }, B

Question 6

Given the program below, what does it print, and at which times?

JavaScript

warmup/js/q6_two_requests.mjs

```
// Question 6. Run: node q6_two_requests.mjs
// get(name, ms) answers name after ms milliseconds: a request, imitated by
// a timer
const get = (name, ms) => new Promise(res => setTimeout(() => res(name), ms));
const t0 = Date.now();
// now(): the time since the start, to the nearest 100 ms
const now = () => Math.round((Date.now() - t0) / 100) * 100 + " ms";

async function main() {
  const pa = get("a", 300);
  const pb = get("b", 100);           // both requests have left
  console.log(await pa, "at", now());
  console.log(await pb, "at", now());
}
main();
```

- (a) b at 100 ms, then a at 300 ms
- (b) a at 300 ms, then b at 300 ms
- (c) a at 300 ms, then b at 400 ms

Question 7

Given the program below, in which `f` awaits a value that is already there, what does it print, in which order?

JavaScript

warmup/js/q7_await_null.mjs

```
// Question 7. Run: node q7_await_null.mjs
async function f(name) {
  console.log(name, 1);
  await null;           // nothing to wait for
  console.log(name, 2);
}
f("x");
f("y");
console.log("main");
```

- (a) x 1, x 2, y 1, y 2, main
- (b) x 1, y 1, main, x 2, y 2
- (c) main, x 1, y 1, x 2, y 2

Question 8

Given the program below, in which each deposit is one line, `balance += await amount(n)`, what does it print? In JavaScript, `balance += e` reads `balance` first, then evaluates `e`, then writes the sum.

JavaScript

warmup/js/q8_plus_await.mjs

```
// Question 8. Run: node q8_plus_await.mjs
// amount(n) answers n after 100 ms: a request, imitated by a timer
const amount = n => new Promise(res => setTimeout(() => res(n), 100));
let balance = 100;

async function deposit(n) {
  balance += await amount(n);
}
// wait until both calls have finished, then print the balance
await Promise.all([deposit(10), deposit(20)]);
console.log(balance);
```

- (a) 130
- (b) 120
- (c) 110

Question 9

Given the deposit of the lecture, which reads the balance, waits for an approval, and writes the new balance, write `depositRepaired` in `warmup/js/q9_deposits.mjs`, in place of `todo(9)`. The repaired deposit still waits for the approval before it changes the balance. Two deposits, of 10 and 20, give 130, whatever the order of their approvals. Then we run `node q9_deposits.mjs`. It prints the balance after the two deposits, in both orders: first for the deposit of the lecture, then for the repaired one.

JavaScript

warmup/js/q9_deposits.mjs

```
// the deposit of the lecture: read, wait, write
async function deposit(account, n, ms) {
  const b = account.balance;
  await approve(n, ms);
  account.balance = b + n;
}

// the same deposit, repaired: it must give 130 in both orders
async function depositRepaired(account, n, ms) {
  todo(9);
}
```

The check: the last two lines, `repaired`, equal approvals and `repaired`, first approval slower, each end in 130.

Part 3: Asynchronous functions in Python (predict on paper, then run)

The lecture asked three questions of every language. Here we ask Python the first two: does a call start the work, and is every `await` a suspension point? In JavaScript the answers are yes and yes (questions 4 and 7). The third question, which thread resumes a suspended computation, has the same answer in both languages: the loop's one thread. The programs are in `warmup/py/`.

Question 10

Given the program below, what does `python3 q10_load.py bare` print?

Python

`warmup/py/q10_load.py`

```
# Questions 10 and 11. Run: python3 q10_load.py bare
#                               then: python3 q10_load.py task
import asyncio, sys

async def fetch_user():
    await asyncio.sleep(0.3)
    return "Ada"

async def load():
    print("A")
    user = await fetch_user()
    print("C", user)

async def bare():
    load()
    print("B")
    await asyncio.sleep(0.5)

async def task():
    asyncio.create_task(load())
    print("B")
    await asyncio.sleep(0.5)

asyncio.run({"bare": bare, "task": task}[sys.argv[1]]())
```

- (a) A, B, C Ada
- (b) a warning that the coroutine `load` was never awaited, then B
- (c) A, C Ada, B

Question 11

Given the same program, what does `python3 q10_load.py task` print?

- (a) A, B, C Ada
- (b) B, A, C Ada
- (c) B only

Question 12

Given the deposit of the lecture in Python, and a program that runs the two deposits twice, first with an `approve()` that has nothing to wait for, then with one that waits for 0 seconds, what does `python3 q12_deposits.py` print?

Python

warmup/py/q12_deposits.py

```
# Question 12. Run: python3 q12_deposits.py
import asyncio

async def nothing_to_wait_for():
    pass

async def really_waits():
    await asyncio.sleep(0)

async def trial(name, approve):
    account = {"balance": 100}
    async def deposit(n):
        b = account["balance"]
        await approve()
        account["balance"] = b + n
    await asyncio.gather(deposit(10), deposit(20))
    print(f"{name:36}", account["balance"])

asyncio.run(trial("approve() has nothing to wait for", nothing_to_wait_for))
asyncio.run(trial("approve() waits, for 0 seconds", really_waits))
```

- (a) 120, then 120
- (b) 130, then 120
- (c) 130, then 130